

Evo-Kernel

个人经验治理与固化层

使用手册

zodyne

2026 年 7 月 24 日

摘要

Evo-Kernel 是位于通用记忆框架之上的「经验治理与固化层」。它以纯文件 + git 作为单一事实源，通过捕获、蒸馏、人审、固化四个环节，把零散的工作经验转化为可检索、可审计、可硬化的约束与技能。

本手册面向日常使用者，覆盖：快速开始、核心 workflow、周期维护、备份恢复、以及使用中的红线与注意事项。

目录

1	Evo-Kernel 是什么	3
1.1	一句话定位	3
1.2	核心能力	3
1.3	架构分层	3
2	目录与状态机	4
2.1	目录角色	4
2.2	经验生命周期	4
3	快速开始	5
3.1	日常检查命令	5
3.2	捕获一个想法	5
3.3	主动召回经验	6
4	核心 workflow	6
4.1	workflow 一：从 capture 到 curate	6
4.2	workflow 二：会话蒸馏	7
4.3	workflow 三：技能固化	8
4.4	workflow 四：批量导入外部资料与技术资料	8
4.4.1	导入的两阶段模型	9
4.4.2	技术资料进哪个区	10

4.4.3	示例：从 Markdown 仓库批量导入	10
4.4.4	研究与借鉴	11
4.5	workflow五：硬约束硬化	12
5	周期维护	12
5.1	每周/每两周做一次	12
5.2	对账与计数	12
5.3	治理动作	13
6	硬约束执行原理	13
6.1	guard 与 hook-guard	13
7	备份与恢复	14
7.1	备份机制	14
7.2	恢复演练	15
8	红线与注意事项	15
8.1	常见误区	15
9	命令速查表	16

1 Evo-Kernel 是什么

1.1 一句话定位

通用记忆框架 (Mem0 / Zep / Letta 等) 解决的是经验的「存储与检索」; Evo-Kernel 解决的是其上没人解决的「治理、固化、可审计」问题。二者是叠加而非替代。

1.2 核心能力

- **捕获**: 零判断入口, 任何想法先丢进 inbox。
- **检索注入**: 按任务自动从 playbook / facts / episodes / principles 召回相关经验。
- **离线蒸馏**: session 结束后对 transcript 切片, 由 Reflector 生成提案, 人审后入库。
- **技能固化**: 经验 → SKILL.md, 可直接被 Claude Code / Pi 挂载。
- **硬约束硬化**: 经验 → guard / hook, 对危险工具调用进行 warn / block。
- **git 可审计**: 每一次 curate / solidify / demote 都是一次可 diff、可 revert 的 commit。

1.3 架构分层

图 1 展示了 Evo-Kernel 的层次结构。上层是 harness (Claude Code / Pi), 下层是纯文件 + git 的内核, 中间通过 hooks 与 CLI 交互。

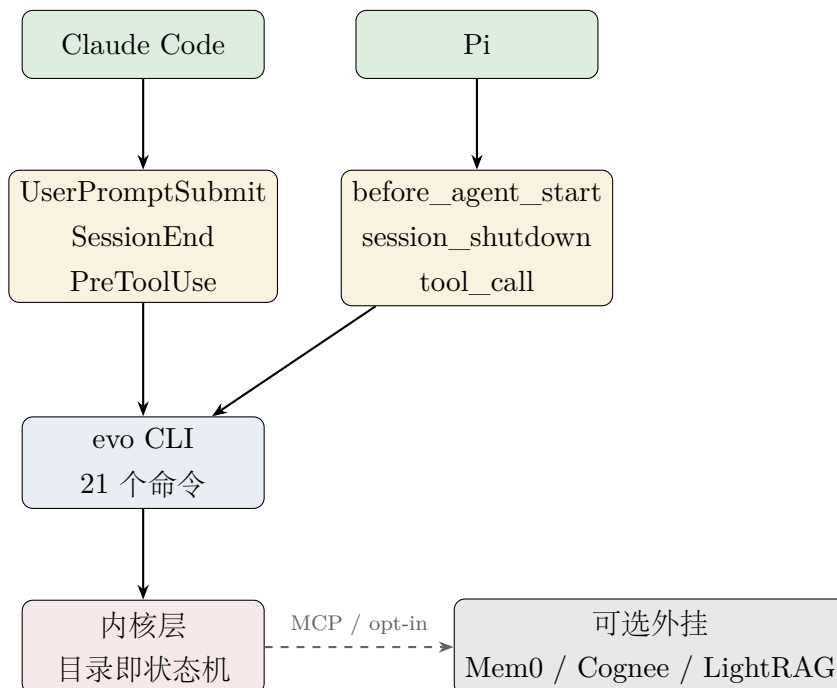


图 1: Evo-Kernel 架构分层

2 目录与状态机

2.1 目录角色

Evo-Kernel 的每个目录都有明确的角色，这是「目录即状态机」的体现：文件在不同目录间移动，即代表经验状态的变化。

表 1: 目录角色与注入权限

目录	角色	是否注入
inbox/	capture 暂存 + 会话登记	否
lessons/	candidate 经验暂存	否
facts/	事实/偏好/环境	是
episodes/	情景记忆：一次任务一份	是
playbook/	策略库，带 helpful/harmful 计数	是
principles/	跨域普适原则	是
skills/	SKILL.md 包 (agentskills.io)	经 skill 注入
ops/archive/	退役/固化存档	否
ops/constraints/	硬约束规则 (JSON)	guard 执行
ops/log/	运行日志 (全部 gitignore)	—
ops/proposals/	reflect/distill 提案	待人审
index/manifest.yaml	派生索引 (可重建)	—

红线

不变量 I2: inbox/ 与 lessons/ 中的条目永远不会被自动注入 context。只有经过人审 curate 进入 playbook / facts / episodes / principles 的条目，才有资格被 recall。否则 capture 这个零审查入口就成了绕过人审的后门。

2.2 经验生命周期

图 2 展示了一条经验从捕获到固化或退役的完整路径。

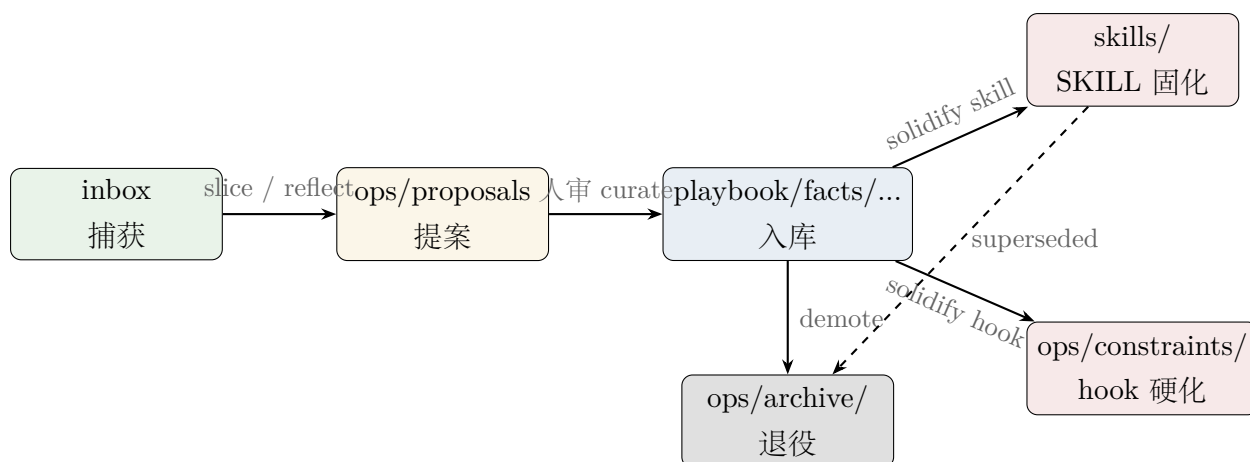


图 2: 经验生命周期状态机

3 快速开始

3.1 日常检查命令

每天开始工作前，先检查系统健康：

```
cd ~/Dev/evo-kernel
./bin/evo doctor
```

期望输出是 OK (N PASS / 1 WARN / 0 FAIL)。唯一常见的 WARN 是 transcript 哨兵比，这是已知飞轮状态，由 reflect 周期跟踪。

3.2 捕获一个想法

任何时候想到一个值得记录的经验、坑或偏好，直接 capture：

```
./bin/evo capture "遇到 Claude Code 权限拦截时，先用
AskUserQuestion 或明确授权，不要偷偷绕过"
```

这会在 inbox/ 下生成一个带时间戳的 markdown 文件。capture 是零判断入口，不需要结构化，不需要 frontmatter。

提示

capture 的目标不是写完美条目，而是不错过任何一闪而过的经验。结构化和人审留给后面的 curate。

3.3 主动召回经验

如果你在写代码或调试，想主动看看有没有相关经验：

```
./bin/evo recall --task "如何安全地删除远程 git 分支"
```

这会输出一段可注入的文本（如果命中）。更常见的场景是 Claude Code / Pi 自动在每次输入时调用 `hook-recall`，你不需要手动跑。

4 核心 workflow

4.1 工作流一：从 capture 到 curate

这是最常见的单条经验入库路径。

Step 1: 把 inbox 里的 raw 文本整理成提案

```
./bin/evo inbox
# 看到一条 capture，打开编辑
# 把它改写成符合 SCHEMA.md 的提案文件，放到 ops/proposals/
```

一个最小提案示例：

```
---
id: avoid-silent-bypass
name: 权限拦截不绕过
type: principle
status: candidate
scope: [workflow]
domains: [claude-code]
triggers:
  - 权限拦截
  - Claude Code 拒绝
  - bypass
evidence:
  helpful: 0
  harmful: 0
verified_by: human
---
```

当 Claude Code 的权限系统拦截一个操作（如删除远程分支、修改系统配置）时，

正确动作是：

1. 用 `AskUserQuestion` 向用户明确请求授权；

2. 或把命令贴给用户，让用户用 `!` 前缀自己执行。

错误动作是：

- 用其他工具绕过权限拦截；
- 把破坏性操作伪装成无害命令。

Step 2: 人审并入库

```
./bin/evo curate --file ops/proposals/avoid-silent-bypass.md --to principles
```

curate 会做三件事：

1. schema 校验 (id / type / status / triggers 必填)。
2. 敏感信息检查 (凭据、密钥、内网地址、PII 不得入库)。
3. 指令样内容审查 (正文必须是陈述式经验，不能是面向未来 agent 的祈使指令)。

通过后，文件被移动到 `principles/`，自动 rebuild manifest、commit、并 push 到 remote。

注意

curate 是唯一的入库口 (不变量 I3)。不要手动把文件拖进 `playbook/` 或 `facts/`，否则绕过了敏感信息和指令样审查。

4.2 工作流二：会话蒸馏

一次完整工作会话结束后，把 transcript 中的经验提炼出来。

Step 1: 确认 session 已登记

Claude Code / Pi 的 session-end hook 会自动写 `inbox/session-refs.jsonl`。你也可以手动登记：

```
./bin/evo session-end --session /path/to/transcript.jsonl --id < session-id>
```

Step 2: 切片

```
./bin/evo slice --session /path/to/transcript.jsonl --ids < session-id>
```

这会输出一个 Markdown 报告，包含：命令 结果对照、写/改文件列表、首条 user、末条 assistant，以及该 session 被注入了哪些经验条目。

Step 3: 生成 reflect 提案

```
./bin/evo reflect --save
```

reflect 会：

- 统计库状态（按 zone、status 分桶）；
- 分析 recall.jsonl（调用数、命中数、空命中任务）；
- 跑 audit，列出治理问题；
- 输出判据对照表（M1/M2/P4 判据、transcript 时效、降级盘点等）；
- --save 时把报告写入 ops/proposals/reflect-< 日期 >.md。

Step 4: 人审并 curate

打开 ops/proposals/reflect-< 日期 >.md，把认可的经验改写成标准提案文件，然后：

```
./bin/evo curate --file ops/proposals/<proposal>.md --to <
  playbook|facts|episodes|principles>
```

4.3 工作流三：技能固化

当一条经验反复有用、值得变成可挂载的技能时：

```
./bin/evo solidify --id avoid-silent-bypass --to skill
```

这会：

1. 在 skills/avoid-silent-bypass/SKILL.md 创建技能包；
2. 把原条目标记为 solidified, 写入 superseded_by: skill:avoid-silent-bypass, 移到 ops/archive/;
3. 自动 rebuild manifest、调用 evo link 同步软链、commit、push。

之后，该 skill 会出现在 ~/.claude/skills/ 软链中，Claude Code 可以加载。

4.4 工作流四：批量导入外部资料与技术资料

如果你已经有大量技术笔记、文档摘录、会议记录或项目沉淀（例如从 Obsidian、Notion、Markdown 仓库导出），不要直接拖进 facts/ 或 playbook/——那样会绕过 curate 的人审、敏感信息检查和指令样审查（违反不变量 I3）。

4.4.1 导入的两阶段模型

借鉴 Zettelkasten 的 “fleeting note → literature note → permanent note” 流程，以及 RAG ingestion pipeline 的 “extract → clean → chunk → enrich → index” 模式，Evo-Kernel 推荐采用 **机器预处理 + 人工审查** 的两阶段模型：

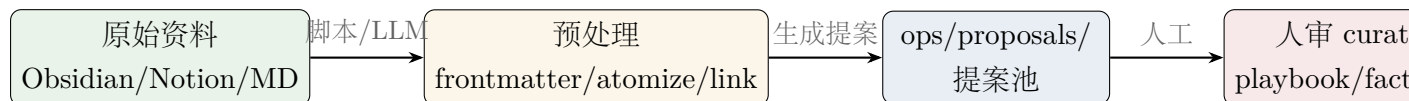


图 3: 外部资料导入的两阶段模型

第一阶段：机器预处理（可自动化）

- **格式规范化**：补全 YAML frontmatter (id、type、status、triggers、domains、scope)，统一文件名；
- **原子化**：把长文档拆成一条一条原子经验（一个知识点/一个检查清单/一个案例一条）；
- **链接转换**：把 wikilink、Notion link 转成 Evo-Kernel 的 id 引用或普通 Markdown link；
- **元数据提取**：用 LLM 或规则自动生成 triggers、domains、scope；
- **暂存提案池**：所有预处理结果放到 ops/proposals/import-< 批次 >/，不进注入通道。

第二阶段：人工审查入库（不可跳过）

- 逐条阅读提案，删除噪音、重复、过时的资料；
- 把认可的内容改写成 Evo-Kernel 的陈述式经验；
- 运行 `evo curate` 进入 facts/、playbook/ 或 episodes/。

红线

为什么不要一轮直接导入？

1. 绕过 curate 的敏感信息检查，可能把凭据、内网地址、客户信息推入 git；
2. 绕过指令样审查，可能把祈使式指令（“总是执行 X”）固化成持久化提示注入；
3. 未经原子化的长文档会降低 recall 精度，大段无关文本稀释 triggers；
4. 噪音和重复条目直接进入 playbook/，会污染注入集、拉低 helpful/harmful 计数质量。

4.4.2 技术资料进哪个区

根据资料性质选择目标目录：

表 2: 技术资料的目标分区

目录	适合内容	示例
facts/	知识点、API 行为、环境配置	“Next.js 15 Server Actions 必须 async”
playbook/	可复用步骤、检查清单、最佳实践	“排查 TypeScript 类型错误三步法”
episodes/	某次具体问题的完整解决过程	“2025-07 部署失败排查实录”
principles/	跨项目/跨技术的通用原则	“优先显式类型而非 any”

4.4.3 示例：从 Markdown 仓库批量导入

假设你有一个 /Notes/ 目录，里面是一堆 Markdown 技术笔记：

```
# 1. 创建提案池
mkdir -p ~/Dev/evo-kernel/ops/proposals/import-2026-07-24

# 2. 用脚本预处理（这里仅示意，实际根据你的笔记结构写）
python3 scripts/import-notes.py \
  --source ~/Notes \
  --target ~/Dev/evo-kernel/ops/proposals/import-2026-07-24 \
  --format evo-kernel
```

预处理后的一个提案文件示例：

```
---
id: ts-type-error-three-steps
name: TypeScript 类型错误排查三步法
type: playbook
status: candidate
scope: [debugging]
domains: [typescript]
triggers:
  - TypeScript 类型错误
  - type error
  - 类型排查
```

```
evidence:
  helpful: 0
  harmful: 0
verified_by: human
---
```

遇到 TypeScript 类型错误时，按以下顺序排查：

1. 先看错误位置的上下文，确认是「声明错误」还是「使用错误」；
2. 检查最近的类型断言、泛型参数、`as` 和 `satisfies`；
3. 若仍无法定位，临时用 `// @ts-expect-error` 标注并附带原因，而不是 `any`。

3. 人审并 `curate`（逐条或分批）

```
cd ~/Dev/evo-kernel
./bin/evo curate --file ops/proposals/import-2026-07-24/ts-type-
  error-three-steps.md --to playbook
```

4.4.4 研究与借鉴

Evo-Kernel 的导入模型参考了两种成熟方案：

- **Zettelkasten**：强调 “fleeting note → literature note → permanent note” 的多阶段处理。临时捕获的笔记应在 24–48 小时内处理，大部分被丢弃，只有经过自己语言重写的原子思想才进入永久库。
- **RAG ingestion pipeline**：典型的生产级流程包括 `extraction` → `cleaning` → `chunking` → `metadata enrichment` → `embedding` → `indexing`。其中最影响检索质量的不是 `chunk` 大小，而是 `chunk` 的语义完整性、上下文线索和元数据治理。

这两种方案的共同点是：**不直接吞入原始资料**，而是经过一个「消化层」——Zettelkasten 靠人工重写，RAG 靠机器分段 + 元数据 + 人工校验。Evo-Kernel 的 `ops/proposals/` + `evo curate` 就是这个消化层。

提示

如果资料量很大，可以先用 LLM 批量生成提案草稿，但 `evo curate` 这一步不要批量自动执行。把 `curate` 当作“最终发布按钮”，保证进入注入通道的每一条都经过人眼确认。

4.5 工作流五：硬约束硬化

当某条经验涉及「会造成实际损失的危险操作」，且 skill/提示层已失效时，可以升为 hook。

```
./bin/evo solidify --id no-rm-rf-root --to hook \\  
  --matcher "rm\\s+-rf\\s+/" \\  
  --match-on command \\  
  --mode warn \\  
  --message "检测到针对根目录的 rm -rf，请人工确认" \\  
  --criteria-confirmed
```

红线

hook 准入四条件（缺一不可）：

1. 可程序化判定：matcher 写得出来、无歧义；
2. 违反代价高昂：不是风格问题，是会造成实际损失的操作；
3. 软层已失效：skill/提示层存在但仍被反复违反（有 harmful 记录佐证）；
4. matcher 覆盖率经复核：明列覆盖/未覆盖的形态与误报语境。

新 hook 必须先 warn 运行一个 reflect 周期，评估误报率后才能升 block。

5 周期维护

5.1 每周/每两周做一次

```
cd ~/Dev/evo-kernel  
./bin/evo index rebuild # 如果 curate/solidify 已经自动 rebuild  
  , 可跳过  
./bin/evo audit # 检查治理问题  
./bin/evo reflect --save # 生成周期报告
```

5.2 对账与计数

helpful/harmful 只能由离线对账单点写（不变量 I4）。主要通道是：

- **自动**：reflect/distill 过程中，Reflector 对每个 session 的注入清单判定四态 (adopted / relevant-unused / irrelevant / misleading)，写入 ops/log/reconcile.jsonl。

- **人工兜底**：如果你明确知道某条经验被采用或误导了，可以手动标记：

```
./bin/evo adopt --ids avoid-silent-bypass # helpful +1
./bin/evo reject --ids outdated-command-pattern # harmful +1
```

之后需要 `evo index rebuild` 把计数汇总进 manifest。

5.3 治理动作

audit 发现的问题通常对应以下动作：

表 3: audit findings 与对应动作			
级别	含义	动作	
HIGH	harmful > helpful	evo reject --ids <id>	或
		evo demote --id <id> --to archive	
MID	superseded 但未归档	evo demote --id <id> --to archive	
MID	lessons 中 status 非 candidate	迁出到正确 zone	
MID	lessons 条目超过 30 天	决定 curate 或 archive	
LOW	playbook / facts / principles 超过 90 天	复验内容是否仍有效	

6 硬约束执行原理

6.1 guard 与 hook-guard

guard 是 Pi 的同步调用路径；hook-guard 是 Claude Code PreToolUse hook 的封装。二者共用 `ops/constraints/*.json` 中的规则。

图 4 展示了 guard 的决策流程。

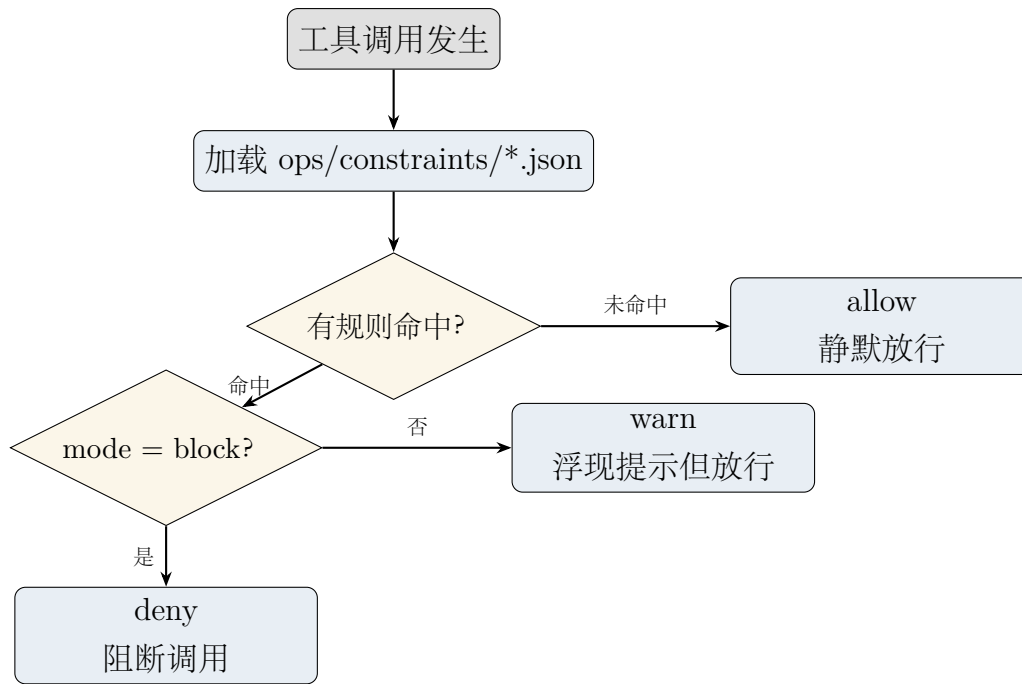


图 4: guard 决策流程

注意

fail-open 原则： guard 自身异常（约束文件损坏、正则非法等）时，一律按 allow 处理。只有「规则正确匹配且 mode=block」才会 deny。这是为了保证经验治理层不会反过来阻塞正常工作。

7 备份与恢复

7.1 备份机制

Evo-Kernel 有两层备份：

1. **主备份：** 每次 `curate` / `solidify` / `demote` 自动 `git push` 到私有 remote（RPO = 一次提交）。
2. **兜底备份：** 定期生成 `git bundle` 到本地异地介质：

```
cd ~/Dev/evo-kernel
git bundle create ~/evo-kernel-$(date +%Y%m%d).bundle --all
```

红线

原始日志不备份：ops/log/*.jsonl 含 prompt 明文，已 gitignore，永不入 remote/bundle。盘损时原始判据数据会丢失，只有 reflect 聚合后的历史保留。这是 §4.4 的诚实权衡。

7.2 恢复演练

每季度做一次：

```
TMPDIR=$(mktemp -d)
cd "$TMPDIR"
git clone git@github.com:zodyne/evo-kernel.git
cd evo-kernel
npm install
./bin/evo doctor      # 此时 harness 挂线项会 WARN/FAIL，属预期
./test/smoke.sh      # 必须 PASS=61 FAIL=0
rm -rf "$TMPDIR"
```

8 红线与注意事项

红线

1. **remote 必须私有**。经验条目含工作语境，公开 remote 直接违反 §4.3 红线。
2. **inbox / lessons 永不注入**。只有 curate 后的分区才有资格进入 recall 通道（不变量 I2）。
3. **helpful/harmful 只由 reconcile 单点写**。禁止实时反馈回路（不变量 I4）。
4. **curate 必须人审**。这是唯一入库口，负脱敏与指令样审查责任（不变量 I3）。
5. **原始日志不入 git**。ops/log/*.jsonl 全部 gitignore（不变量 I4 / §4.4）。
6. **hook 升 block 前必须过 warn 观察期**。且需人工复核 matcher 覆盖率 (§8)。
7. **不要多机并写**。remote 仅备份，第二台机器只读消费；多机并写会 ID 碰撞、计数合并未定义 (§4.1)。

8.1 常见误区

- 「我把文件拖进 playbook/ 就行了吧？」不行。curate 是唯一能写入 playbook/-facts/episodes/principles 的入口。

- 「recall 空命中很多，是不是该上 M1 FTS5？」 M1 判据是「注入精度连续 2 周期 $< 50\%$ 且 $\text{recall} \geq 200$ 次」。现在 recall 才约 50 次，数据不足，做了也是过早优化。
- 「transcript 哨兵比高，是不是 bug？」不是 bug。Claude Code 会按策略清理 transcript 文件，登记与蒸馏之间存在腐烂窗口。由 reflect 周期跟踪，不是靠实施解决。
- 「我想在另一台机器上也 curate。」暂时不要。多机并写未设计，只读消费可以。

9 命令速查表

表 4: Evo-Kernel 21 命令速查

命令	用途	典型场景
capture "..."	零判断捕获	随时记录想法
recall --task "..."	主动检索	手动查经验
hook-recall	harness 注入入口	由 hooks 自动调用
session-end	手动登记会话	hook 未触发时
hook-session-end	自动登记入口	由 hooks 自动调用
adopt --ids	人工 +helpful	明确知道某条被采用
reject --ids	人工 +harmful	明确知道某条误导
curate --file --to	人审入库	核心流程
slice --session	transcript 切片	蒸馏前准备
index rebuild	重建 manifest	curate 后自动做
candidates	列出候选条目	agentic 粗筛
get --ids	取全文	agentic 精筛
reflect --save	周期复盘	每周/每两周
audit	治理检查	配合 reflect
inbox	待处理清单	查看 capture 和 refs
solidify --id --to skill	技能固化	经验变 SKILL.md
solidify --id --to hook	硬约束注册	危险操作防护
demote --id --to archive	退役	经验过期/有害
guard / hook-guard	约束执行	由 harness 调用
link	同步 skills 软链	solidify 后自动做
doctor [--full]	部署自检	每日/每次重大变更后